

A Comparative Empirical Study of FIDO2 and Its Predecessors

Anderko Máté
Óbuda University
mate.anderko@stud.uni-obuda.hu

Abstract—The FIDO2 standard, introduced by the FIDO Alliance and the World Wide Web Consortium (W3C) in 2018, it has been hailed as the “Kingslayer of User Authentication”. This paper presents the implementation of a FIDO2 + WebAuthn authentication system and shows an analysis on different user authentication philosophies. The analysis assesses a FIDO2, a 2FA and a password only system on their resistance to common security threats using the STRIDE threat modeling framework and evaluates their performance. The objective is to determine how adopting FIDO2 influences the security and system resource requirements.

1. Introduction

Since the dawn of computing the weakest link in most information systems has been the people creating and using them. We choose easy to remember password that might be prone to dictionary attacks, then we over-complicate them just to forget them, so to only have to remember a couple passwords we reuse them across sites, also we easily fall for social engineering attacks, like phishing. Sufficiently long, unique, and complex passwords with proper rate limiting remain impractical to brute-force, so compromises overwhelmingly exploit human factors and credential leaks rather than cryptographic weakness.

To mitigate the weaknesses of a password only environment one-time codes (SMS/TOTP) are introduced, this is 2 or Multi Factor Authentication, which is meant to reduce the risk of falling for credential stuffing attacks and make phishing more complicated. 2FA doesn’t address the core issue of legacy authentication though.

FIDO2/WebAuthn addresses the limitations at last by replacing shared secrets with origin-bound public-key cryptography and local user verification (biometrics or a device PIN). Private keys never leave the authenticator, and the server stores only public keys, eliminating password reuse and greatly reducing the value of stolen databases. Because assertions are tied to the requesting origin, generic phishing and OTP-relay attacks fail by design. In practice, passkeys (discoverable credentials) further streamline the flow across platforms using platform sync or roaming security keys.

This paper adopts that progression from passwords to 2FA to FIDO2. In the coming section we will walk through

- Mapping common vulnerabilities
- Implementing a fully passwordless FIDO2 system
- Creating a plan for a STRIDE based analysis
- Testing the systems for vulnerabilities
- Comparing the systems in terms of performance and security.

2. Similar and Related Work

This section reviews research on the security, usability, and adoption of FIDO2, also shows a comparison with this work.

2.1. Formal and Theoretical Analyses

Barbosa and their team [1] offers a formal model of FIDO2 and, through theoretical analysis, show resistance to credential-phishing and replay attacks. In contrast, the present work examines a deployed FIDO2 stack with STRIDE, emphasizing practical security implications and performance (latency, memory, compute).

2.2. Empirical and Usability Studies

Lyastani and their team [2] created an empirically FIDO2’s usability analyses. The study shows significant usability gains and phishing resistance compared to passwords and 2FA. Their work relied on user studies and perception metrics rather than system-level measurements.

Unlike these human-centered, behavioral studies, this work isolates the technical performance and security aspects of FIDO2.

2.3. Positioning of This Work

Key contributions:

- 1) Operational implementation of FIDO2/WebAuthn alongside password-only and 2FA baselines.
- 2) Structured STRIDE analysis, grounded in Barbosa’s theoretical model.
- 3) Empirical performance results—latency, memory footprint, computational load—that complement usability-centric studies (e.g., Lyastani).

Unlike work centered on formal guarantees or user experience, this study examines how adopting FIDO2 shifts real-world security posture and runtime efficiency.

3. Common Authentication Vulnerabilities

Bad actors find the most creative ways to attack authentication systems, these are called attack vectors. The goal of this section is to outline how the vulnerabilities work, which the paper will address in later sections, and also to shine a light on why traditional password or even basic MFA schemes struggle against them.

3.1. Weak Passwords and Brute-Force Attacks

Users often choose simple passwords, making them easy targets for brute-force attacks [3]. A simple brute-force attack tries to crack the user's password by trying random character combinations. Rate limiting and sufficient password length makes the attack redundant. A dictionary attack is a more sophisticated version of a brute-force attack, where attackers use common wordlists, lists of frequently used passwords or they systematically guess combinations to find the password. In the case of brute-force not cryptographic weaknesses, but bad password hygiene and the site's lack of password complexity enforcement leads to the compromise.

3.2. Credential Stuffing and Password Reuse

Credential stuffing exploits password reuse across websites. In the event of a databreach the user's password may get leaked, this creates an opportunity for bad actors, who then use the credentials on other sites to try to gain access to other accounts.[4]. The process can be well automated, thus the reuse of credentials turns one breach into many.

3.3. Social Engineering Attacks

Many attacks rely on human gullability instead of technical flaws. Phishing, SIM swap, and push fatigue attacks are such attacks.

3.3.1. Phishing and OTP Relay

Phishing deceives users into revealing credentials on fake websites, that mimic the working of the site the user is familiar with. Attackers can proxy the login and capture OTP codes in real time, bypassing traditional 2FA, this is commonly referred to as a real-time phishing proxy. OTP capturing is Adversary-in-the-Middle type attack[5].

3.3.2. SIM Swap Attacks

The way SMS based MFA works is that the user gets an SMS containing a secret code, generated by the server, this while may seem bulletproof requires complete trust in the user's telecommunication service provider. The attackers may be able to impersonate victims and hijack their phone number (with the service provider unknowingly complicit), thus gaining control of SMS-based OTPs[6].

3.3.3. MFA Push Fatigue

Push-based MFA can be exploited by flooding users with repeated prompts until they approve one, often by mistake or frustration [7].

3.4. Server-Side Data Breaches

When there is a data breach in the authentication server's database, password hashes or shared secrets (like TOTP seeds) can get exposed, and then the attackers can crack these sensitive information offline to gain access to the accounts [8].

3.5. Adversary-in-the-Middle tampering

Interception and modification of traffic between the client and server by a third party falls into the category of tampering. One would say, that TLS provides transport security, but by itself it does not prevent manipulation if it gets compromised or stripped [9].

3.6. Denial of Service via Account Lockout

Sometimes security measures themselves can be weaponized against the users or the service, attackers trigger account lockouts by repeatedly submitting invalid credentials for a target user. This exploits the lockout policy, while it keeps the user safe from brute-force attacks, it can lead to them not being able to access their account [10].

4. How FIDO2 Works

This section offers a quick introduction into the FIDO2 ecosystem. FIDO2 can be described by combining the W3C WebAuthn API with the CTAP protocol by FIDO [11], it's a public key based origin-bound authentication method.

4.1. Core Roles

- 1) **Relying Party (RP):** The service that requests authentication and stores the public key.
- 2) **Client:** The browser which mitigates communication between the RP and the authentication hardware.
- 3) **Authenticator:** The device (Like a Yubico key or Android Strongbox) that generates and stores private keys and asserts identity [11].

4.2. Cryptographic Mechanism

FIDO2 relies on public-key cryptography, there is no exchange of shared secrets like in a password based environment.

- **Registration:** The authenticator generates an asymmetric key pair inside a Secure Element. The **private key** never leaves the device, while the public key is sent to the RP.
- **Authentication:** The RP issues a random challenge, then using the private key the authenticator signs this challenge, this creates a **digital signature** proving possession of the credential and verifies the origin without ever revealing the private key over the network [11].

4.3. Safety & Security Architecture

There are multiple security features that make FIDO2 a strong standard [11], like:

- **User Verification:** For the authenticator to proceed with "User Presence" is not enough, meaning that by stealing the hardware attackers won't be granted access, to login "User Verification" is needed, this could be biometrics or other method, like using a PIN.
- **Origin Binding:** The authenticator checks where the request comes from, making the standard phishing proof, because this origin can not be tampered, since that would break the digital signature.
- **Attestation:** During setup, the authenticator shows a digital manufacturing certificate to the server, for legitimacy verification

5. Implementation

This section talks about the components of, and how they interact in the prototype, that we will run our STRIDE testing and performance analysis on, we will also talk about the packages used in the backend, this is important to reflect on in when evaluating the results.

5.1. Backend Packages

We use the backend packages, listed in Table 1. Python's FastAPI framework handles requests, while 'pydantic' ensures data integrity. For our database we use PostgreSQL (via 'psycopg2-binary') as it is FOSS and is widely used in the industry, to link our database to our application we use 'sqlalchemy', which serves as an ORM layer, so we can interact with the database in a type-safe way. The package 'redis' stores state objects, like WebAuthn challenges. Session management is done by 'pyjwt' package, it has the job of issuing JSON Web Tokens upon successful login.

We implement different authentication methods:

- **Password:** For hashing the user's password we use the 'bcrypt' library, configured with a work factor of 12 salt rounds to align with OWASP recommendations [8], according to the guidelines, this should ensure that stored credentials remain resistant to rainbow table and brute-force attacks while maintaining acceptable server performance (200ms).
- **2FA (TOTP):** Besides the password management we also had to implement a 2FA system, using the Time-based One-Time Password algorithm. We chose the 'pyotp' library, supported by 'qrcode' for provisioning, we follow RFC 6238 [12] during the implementation, we used HMAC-SHA1 based OTP, requiring a 6 digit code.
- **FIDO2/WebAuthn:** For credential registration and authentication we use the official 'fido2' library with 'cbor2' for binary data decoding.

6. STRIDE Threat Modelling Framework

When evaluating the security of the authentication methods, we applied a framework that allow us to analyse them and compare their security, the tool of our choosing was the STRIDE threat modeling framework.

TABLE 1. BACKEND PYTHON PACKAGES AND THEIR ROLES

Package	Role
'fastapi'	Web framework providing REST API
'uvicorn'	ASGI server, runs FastAPI
'fido2'	FIDO2 credential handling
'sqlalchemy'	ORM to access DB
'psycopg2'	PostgreSQL database adapter
'pydantic'	Data validation management
'redis'	State store for challenge data
'pyjwt'	JWT token generation and verification
'cbor2'	CBOR encoding/decoding binary data
'bcrypt'	Password hashing
'pyotp'	TOTP code generation and verification
'qrcode'	QR code generation for TOTP

It creates an abstraction of real-world attacks by grouping them into the following types:

STRIDE categories:

- **Tampering:** Tampering attacks mainly focus changing on the message content in transit, while staying undetected.
- **Repudiation:** Repudiation can be defined by successfully stating that an action was not performed by the user.
- **Information Disclosure:** Information disclosure is the category, where sensitive data gets exposed to actors with no permission to access it.
- **Denial of Service:** In a DoS attack one or more user loses access to the service. This may happen by flooding the server with requests or by getting users locked out of their accounts.
- **Elevation of Privilege:** Elevation of privilege happens when bad actors get admin rights. This may happen by stealing credentials or adding themselves to more privileged groups.

7. STRIDE Analysis

This section aims to explain how each system should fare in each STRIDE category.

7.1. Spoofing

- **Password-only:** Password-only systems are the most vulnerable to phishing attacks and credential stuffing, since they entirely rely on the user's better judgement, there is no origin binding, or relying on a safe third party.
- **2FA:** 2FA creates a much stronger barrier against phishing attacks, since it provides another wall of defence, in the form of a second factor. Depending on the type of this factor attackers can use different ways to bypass it mostly via Real Time Phishing (Relay Attacks), or Sim Swapping.
- **FIDO2/WebAuthn:** FIDO2's origin binding ensures that even if an attacker manages to phish the user, they can't redirect the user to a fake site, since the system would not accept the authentication request from a foreign RP ID.

7.2. Tampering

- **Password-only:** Usually password only systems offer no protection against message tampering beyond TLS. If TLS is compromised or misconfigured, an attacker can modify login requests or responses.
- **2FA:** 2FA TOTP systems also rely primarily on TLS for tampering protection. OTP codes can be intercepted during transmission, for example in a relay attack where the attacker uses a proxy to phish the user's password and intercept the OTP code, then uses it to authenticate.
- **FIDO2/WebAuthn:** Since FIDO2 is a digitally signed challenge-response system, modification to the challenge or response invalidates the signature, making it virtually tamper-proof even if TLS is compromised.

7.3. Repudiation

- **Password-only:** There is no repudiation protection in a barebones password system, anyone with the password can authenticate and there is no way to link a particular login to a device or authenticator. Even logging can be manipulated since we can not be sure about the user's identity.
- **2FA:** 2FA can provide some repudiation protection by logging OTP usage, but these logs can be modified by attackers who manage to gain high enough privileges. Proving the who performed the authentication is difficult in these systems, since cryptographic proof of which device generated the OTP code is not provided by the standard.
- **FIDO2/WebAuthn:** Since the private keys are stored on the authenticator, and these encapsulated systems manage them FIDO2 offers strong repudiation protection, because other devices can not copy this information. The signature counter also helps detect credential cloning attempts.

7.4. Information Disclosure

- **Password-only:** Password systems are the most vulnerable to information disclosure, since breaches can expose password hashes that can be later used for offline cracking, the passwords can also be intercepted during transmission.
- **2FA:** TOTP reduces information disclosure risk compared to passwords. The TOTP secret is stored server-side and never transmitted, but if the database is compromised bad actors may be able to generate valid codes.
- **FIDO2/WebAuthn:** In a FIDO2 system only public keys are stored server-side, and private keys never leave the authenticator, meaning that an attacker would not be able to access accounts and gain access to sensitive information.

7.5. Denial of Service

- **Password-only:** There are multiple DoS attack vectors for password systems, and bad actors might use brute-force to overwhelm the whole server, but there are certain types of DoS attacks that focus on attacking a single user via the server's security features like lockout mechanisms, where attackers might trigger lockouts by repeatedly attempting incorrect passwords, preventing legitimate users from accessing their accounts.
- **2FA:** 2FA systems inherit the vulnerability of password-only systems, the lockout mechanism may shift to the

OTP code level, but the attackers can also exhaust backup codes, flood users with OTP requests.

- **FIDO2/WebAuthn:** Passwordless authentication is resilient to DoS attacks, its biggest vulnerability is the hardware doing the authentication, like wiping data from the authenticator in a CTRAPS attack, which factory resets private keys. On the server side there is no incentive to penalise users with lockouts, since there is no password for the attackers to guess.

7.6. Elevation of Privilege

- **Password-only:** Since administrative accounts without the proper precaution can fall victim to a multitude of attacks, bad actors can gain initial access to administrative accounts and elevate their privileges.
- **2FA:** 2FA and password only systems are similarly vulnerable to privilege elevation attacks, even though the additional factor makes them more difficult, with the proper safety measures in place seemingly impossible for attackers to gain access, the elevation of privilege is still possible with the aforementioned techniques.
- **FIDO2/WebAuthn:** For FIDO2 elevation of privilege attacks do not usually target the cryptographic protocol, but they may try to register their FIDO keys to a high privilege account, granting them access to rights they should not have.

8. Empirical STRIDE Security Testing

This section presents the empirical validation of the STRIDE threat analysis described in Section 7. To verify the theoretical security properties of the three authentication methods—password-only, TOTP-based 2FA, and FIDO2/WebAuthn—we developed an automated security testing suite. The objective was to confirm the presence of expected vulnerabilities in legacy systems and validate the security protections provided by FIDO2. The methodology: The security tests were implemented in the `pytest` framework. To ensure the results are reliable and reproducible, the testing followed the key points of:

- **Environment Isolation:** Each test runs in a separate, clean environment. We used in-memory databases for every test case. This prevents interference between tests and ensures consistent results.
- **Full System Testing:** The test - backend interaction is achieved by using an asynchronous client (`httpx`) in an end-to-end manner. This simulates how a bad actor would interact with the system.
- **Hardware Simulation:** For FIDO2 tests, we created a `MockAuthenticator` software component, that was used for cryptographic signing and key management, bypassing the requirement for a WebAuthn authenticator Hardware, allowing us to generate both valid and invalid authentication data to test the server's response to various attack scenarios.

8.1. Spoofing

- **Password-only:** Impersonation using only a password was straightforward via a credential stuffing attack. When the test case used a simulated

“leaked” password from a different site, the server accepted the attempt. This confirmed that the password was not origin-bound, making the account vulnerable if credentials are reused.

- **2FA:** To test TOTP, we executed a relay attack, the server accepted the code, showing that TOTP does not inherently prevent relay attacks.
- **FIDO2/WebAuthn:** We simulated a phishing attempt by using `MockAuthenticator` to log in with an incorrect RP. The server correctly rejected the login because the cryptographic signature did not match our expected origin, proving its resistance to phishing.

8.2. Tampering

- **Password only and 2FA:** We simulated a compromised TLS layer and we were able to modify the user’s credentials to that of an attacker, without the server detecting it.
- **FIDO2/WebAuthn:** The FIDO2 suite uses digital signatures, when we tried modifying it along the data payload in a valid FIDO2 response, the server rejected the request. This confirmed that FIDO2 systems are able to verify the integrity of the data, independent of the network layer.

8.3. Repudiation

- **Password only and 2FA:** Since the server knows the secrets, database administrators could forge log entries tied to users. There is no proof that ties an action to a specific user.
- **FIDO2/WebAuthn:** A digital signature consists of hash encrypted with the user’s unique private key, the server stores this signature, which provides strong evidence that the specific user authorized the action. Our test confirmed that the server can verify the signature, and that it is unique to the user.

8.4. Information Disclosure

- **Password only and 2FA:** For password systems, the test retrieved password hashes, which can be cracked offline. In the case of the 2FA TOTP system, the test retrieved the shared secrets, allowing an attacker to generate valid codes.
- **FIDO2/WebAuthn:** For FIDO2, the test confirmed that the database contains only public keys. We verified that these keys cannot be used to authenticate, meaning a database breach does not compromise user accounts.

8.5. Denial of Service (DoS)

- **Password only and 2FA:** We sent many invalid password attempts to a user account. The system locked the account to prevent guessing.
- **FIDO2/WebAuthn:** We sent invalid requests to the FIDO2 endpoint. The server rejected the invalid signatures efficiently. Because there is no shared secret to guess, the system does not need to lock accounts as aggressively, making it more resilient to this type of attack.

8.6. Elevation of Privilege

- **Password only and 2FA:** We simulated a phishing attack against an administrator. For password and TOTP systems, obtaining the credentials allowed full access to the admin account.
- **FIDO2/WebAuthn:** For FIDO2, we showed that knowing the admin’s username was not enough. The test confirmed that authentication failed without the physical security key, proving that remote attackers cannot elevate privileges even if they have user information.

9. Performance and Resource Analysis

This section presents an empirical evaluation of the performance and resource consumption of the implemented authentication methods. We’ve seen that FIDO2 is the our most secure technology, but does it mean it will lack the performance of legacy systems? Now we will delve into the end-to-end latency, server-side CPU overhead, and database load, comparing how the three implemented methods fared. The tests were performed on an 11th Gen Intel® Core™ i5-11400H and 32 GiB of RAM

9.1. Latency Analysis

We analyze the latency of registration and login operations, excluding the time spent on the client device, research indicates this introduces a delay of 50–200 ms depending on the device [13]. We think this is a more accurate representation of the server-side performance. The tests are conducted in an isolated environment to minimize external factor and lessen the scope of testing.

9.1.1. Registration Performance

Figure 1 shows the registration latencies for each method.

Password Registration (≈640 ms): As we have mentioned in 5, for the bcrypt hashing to be secure we have to use at least 10 salt rounds, for our implementation we use 12 salt round, which means that the algorithm runs 12 times and takes about 640 ms to hash a password.

2FA Setup (≈670 ms): This method inherits the latency of password registration, and also adds a negligibly small overhead for secret generation, thus bcrypt step remains the bottleneck.

FIDO2 Registration (≈15 ms): Since keygeneration happens on the client hardware, the server only has to verify the signature, not require to do any computationally expensive operations, like the aforementioned hashing [2].

We can see that all the latencies are still within the realm of acceptability, but FIDO2 might be a better alternative at high loads, like at opening days of ticket sales for large scale events.

9.1.2. Login Performance

Login performance proved much similar to registration, we still saw a huge gap between the legacy methods

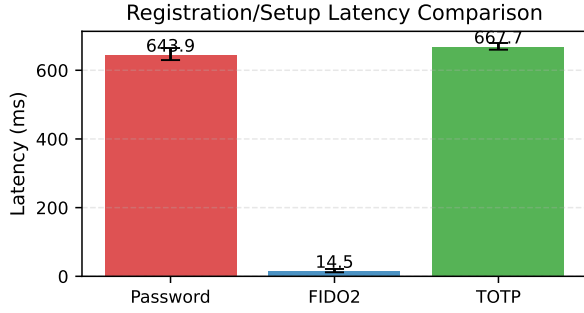


Figure 1. Registration latency comparison. FIDO2 outperforms legacy methods by offloading cryptography to the client.

and FIDO2 since password hashing still had to be done in order to log the user in

9.2. Database Load

Database query analysis is shown in Figure 2.

Registration: Password and 2FA flows often require less steps, than FIDO2 based systems, because FIDO2 also needs to check for existing credentials. This only means 3 additional queries keeping it reasonable.

Login: All methods are highly optimized. They typically require only a single lookup. Database load is not a differentiating factor for login.

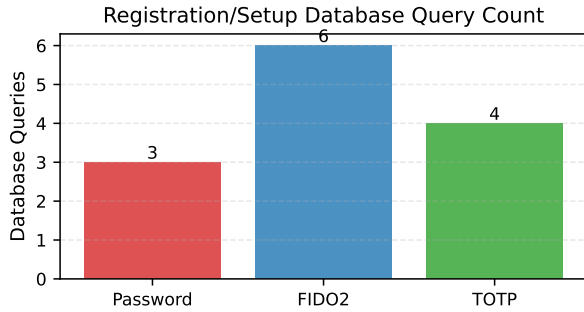


Figure 2. Database query count comparison for registration flows.

10. Conclusion

This paper presented a comparative analysis of Password-only, 2FA (TOTP), and FIDO2/WebAuthn authentication mechanisms, evaluating them using empirical STRIDE threat analysis, and performance metrics.

Our security analysis confirms that while 2FA improves upon password-only systems by adding a layer of defense, though it only acts as a patch and doesn't address the underlying issues, FIDO2 on the other hand fundamentally addresses these vulnerabilities

FIDO2 is not only the safest, but also the show the best performance by offloading heavy cryptographic operations to the client device(that are usually under-used nowadays), reducing server-side processing time, resulting in lower latency and reduced CPU overhead.

In conclusion, FIDO2 represents a significant leap forward in authentication technology, it may be rightfully called the Kingslayer of authentication methods, even with the downside of being reliant on the client device.

References

- [1] M. Barbosa *et al.*, “Provable security analysis of fido2,” in *Information Security Theory and Practice (WISTP 2021)*. Springer, 2021. [Online]. Available: https://doi.org/10.1007/978-3-030-84252-9_5
- [2] S. Lyastani *et al.*, “Is fido2 the kingslayer of user authentication?” in *IEEE Symposium on Security and Privacy*, 2020, accessed: 2025-11-23. [Online]. Available: <https://trust.cispa.saarland/publication/lyastani-20-sp/lyastani-20-sp.pdf>
- [3] E. Ma, S. Hasama, E. Lumba, and E. Birrell, “Prospects for improving password selection,” *arXiv preprint arXiv:2201.01350*, 2022. [Online]. Available: <https://arxiv.org/abs/2201.01350>
- [4] A. Nisenoff, M. Golla, M. Wei, J. Hainline, H. Szymanski, A. Braun, A. Hildebrandt, B. Christensen, D. Langenberg, and B. Ur, “A two-decade retrospective analysis of a university’s vulnerability to attacks exploiting reused passwords,” in *Proc. USENIX Security Symposium*, 2023. [Online]. Available: <https://www.usenix.org/system/files/usenixsecurity23-nisenoff-retrospective.pdf>
- [5] M. Käsemodel, V. Winterhalter, F. Heisel, F. Alt, and B. Pfleging, “BlueTOTP: Designing phishing-resistant and user-friendly two-factor authentication,” in *Proc. IFIP INTERACT 2025 (LNCS 16110)*. Springer, 2025, pp. 611–634. [Online]. Available: https://doi.org/10.1007/978-3-032-05005-2_31
- [6] K. Lee, B. Kaiser, J. Mayer, and A. Narayanan, “An empirical study of wireless carrier authentication for SIM swaps,” in *Proc. 16th Symposium on Usable Privacy and Security (SOUPS)*. USENIX Association, 2020, pp. 61–79. [Online]. Available: <https://www.usenix.org/conference/soups2020/presentation/lee>
- [7] Cybersecurity & Infrastructure Security Agency (CISA), “Implementing phishing-resistant mfa (fact sheet),” <https://www.cisa.gov/sites/default/files/publications/fact-sheet-implementing-phishing-resistant-mfa-508c.pdf>, 2022, accessed: 2025-11-23.
- [8] OWASP Foundation, “Password storage cheat sheet,” https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html, 2024, accessed: 2025-11-23.
- [9] —, “Manipulator-in-the-middle attack,” https://owasp.org/www-community/attacks/Manipulator-in-the-middle_attack, 2024, accessed: 2025-11-23.
- [10] National Institute of Standards and Technology, “Digital identity guidelines: Authentication and lifecycle management (nist special publication 800-63b),” U.S. Department of Commerce, Tech. Rep., 2017, accessed: 2025-11-23. [Online]. Available: <https://doi.org/10.6028/NIST.SP.800-63b>
- [11] “Fido2 specifications,” <https://fidoalliance.org/specifications/>, 2021, accessed: 2025-11-23.
- [12] D. M’Raihi, J. Rydell, F. Hoornaert, S. Machani, and P.-C. Hsiang, “TOTP: Time-Based One-Time Password Algorithm,” RFC 6238, May 2011. [Online]. Available: <https://www.rfc-editor.org/info/rfc6238>
- [13] L. Würsching, S. Putz, M. Häslar, and M. Hollick, “Platform vs. roaming authentication on smartphones,” in *CHI Conference on Human Factors in Computing Systems*, 2023, accessed: 2025-11-23. [Online]. Available: https://peasec.de/paper/2023/2023_WuerschingPutzHaeslerHollick_PlatformvsRoamingAuthenticationonSmartphones_CHI.pdf